

ТАБЛИЧНЫЕ ТИПЫ

СОДЕРЖАНИЕ

1	ТАБЛИЧНЫЕ ТИПЫ	1
1.1	ТАБЛИЦА С ФИКСИРОВАННОЙ РАМКой	1
1.2	СТРУКТУРА РАМКИ ТАБЛИЦЫ	2
1.2.1	Заголовок	3
1.2.2	Часть элементов	4
1.3	ТАБЛИЧНЫЙ РЕЖИМ	6
1.3.1	Массив	6
1.3.2	Кольцевой массив	8
1.3.3	Стек (поддерживается еще не всеми функциональными блоками)	9
1.3.4	Очередь (поддерживается еще не всеми функциональными блоками)	10
1.4	АДРЕСАЦИЯ ВНЕШНЕЙ ТАБЛИЦЫ	11
1.4.1	Передача данных	11
1.4.2	Первый элемент	11
1.4.3	Функционирование	13
1.5	ЭЛЕМЕНТЫ ЗАГОЛОВКА	14
1.6	ПРИМЕР КОНФИГУРАЦИИ 1	18
1.7	ПРИМЕР КОНФИГУРАЦИИ 2	22
1.8	ССОТ - ФУНКЦИОНАЛЬНЫЙ БЛОК УСЛОВНОГО КОПИРОВАНИЯ ТАБЛИЦЫ	25
1.8.1	Символьное представление	25
1.8.2	Функциональное описание	25
1.8.3	Инициализация	28
1.8.4	Пример конфигурации	29
1.8.5	Структура данных	29

1 ТАБЛИЧНЫЕ ТИПЫ

1.1 ТАБЛИЦА С ФИКСИРОВАННОЙ РАМКОЙ

Эта таблица является структурой с фиксированной рамкой, определенная типом и содержащая таблицу, структура которой может быть изменена на стадии конфигурации или выполнения.

Тип таблицы с фиксированной рамкой определяет размерность рамки. Тип также определяет тип элементов, как в рамке, так и в таблице внутри рамки. Эти типы не могут быть изменены в процессе конфигурации или выполнения. Каждая требуемая комбинация типов элементов и размерности должна существовать как отдельный тип. Совокупность типов каждой версии metsoDNA CR определяет то, какие табличные типы содержит данная версия.

Имя табличного типа указывает как тип элементов, так и размерность рамки таблицы. Формат имени имеет вид `typ_ijk`, например, `ana_9` или `cha_536`.

Первая часть имени табличного типа (`typ`) является аббревиатурой типа элементов. Аббревиатуры (`typ` и соответствующий тип элементов) могут быть следующими:

Основные типы:

```
cha = char
i8 = int8
i16 = int16
i32 = int32
u8 = uns8
u16 = uns16
u32 = uns32
b8 = bool8
b16 = bool16
flo = float
fls = fails
```

Общие типы:

```
ana = ana
bin = bin
ins = ints
inl = intl
bne = binev
ise = intsev
```

Типы приложений:

```
bo = bo
```

Вторая часть имени табличного типа (ijk) обозначает размерность рамки таблицы, каждая буква обозначает одну из размерностей. Рамка таблицы может иметь размерность не более 3-х. Параметры i, j и k представляет собой кодированные экспоненты, указывающие степень двух. Коды экспонент и соответствующие им значения размерностей представлены ниже (код и размерность):

```
0 = 1
1 = 2
2 = 4
3 = 8
4 = 16
5 = 32
6 = 64
7 = 128
8 = 256
9 = 512
a = 1024
b = 2048
c = 4096
d = 8192
e = 16384
```

Например, `ana_9` - представляет собой одномерную таблицу. Тип элементов есть `ana`, а размерность рамки 512. `Cha_536` представляет собой трехмерную таблицу. Тип элементов есть `char`, а размерность рамки 32x8x64.

1.2 СТРУКТУРА РАМКИ ТАБЛИЦЫ

Как структура данных, табличная структура состоит из двух частей: заголовка (элемент `:header`) и части элементов (элемент `:elem`). Каждая таблица рамки имеет тот же самый тип (таблицу), что и заголовок, независимо от размерности и типа элементов рамки. Рамки таблицы, имеющие разные размерности и типы элементов, создают путем использования различных частей элементов и разных значений, присваиваемых по умолчанию в заголовках. Примером структуры рамки таблицы `ana_9` является:

`ana_9`

- элемент заголовка **`header`** типа **`table`**
- элемент **`elem(512)`** типа **`ana`**

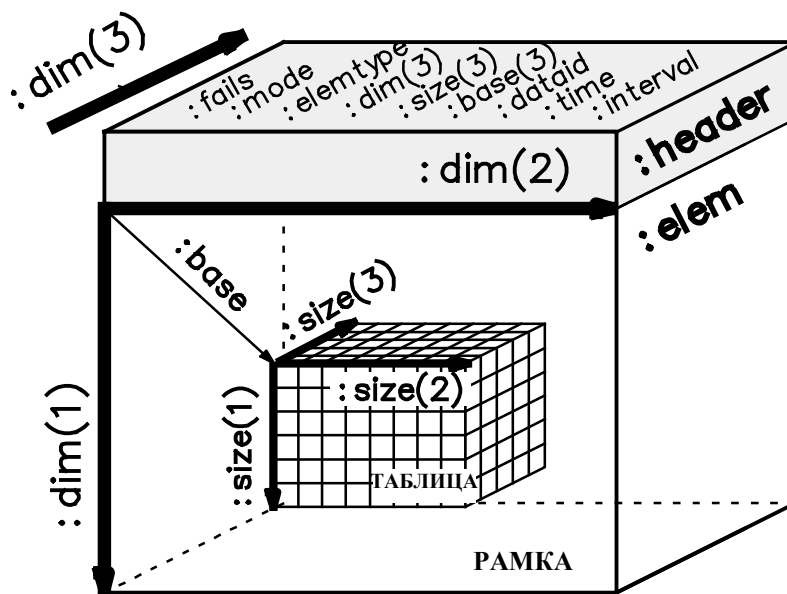


Рисунок 1 Структура рамки таблицы

1.2.1 Заголовок

Тип заголовка (:header) рамки таблицы есть table. Он состоит из структурных данных рамки (:elemtype и :dim) и таблицы (:mode, :size и :base). Он также содержит отдельные скалярные значения (:fails, :dataid, :time и :interval), описывающие таблицу. Структура заголовка (типа table) следующая:

table

- элемент **fails** типа **fails**
- элемент **mode(16)** типа **char**
- элемент **elemtype(8)** типа **char**
- элемент **dim(3)** типа **intl**
- элемент **size(3)** типа **intl**
- элемент **base(3)** типа **intl**
- элемент **dataid** типа **intl**
- элемент **time** типа **binev**
- элемент **interval** типа **ana**

Смысловое значение каждого элемента заголовка будет описано в дальнейшем более подробно.

Данные из заголовка можно считать и записать путем обращения к соответствующему элементу, например, pr:PM6:BW:header:size(3) или pr:PM6:BW:header:dataid или путем обращения ко всей рамке таблицы, например, pr:PM6:BW.

Значения элементов заголовка должны быть совместимы одно с другим и с элементной частью. Обычно совместимость обеспечивается на стадии конфигурирования путем формы заполнения табличного символа и во время выполнения программы при помощи функционального блока, который может управлять таблицами. Если таблицы будут сконфигурированы или обработаны во время выполнения программы объектами, которые не понимают таблицы (например, тип символа "any" или скалярные функциональные блоки), совместимость должна быть достигнута другим способом.

1.2.2 Часть элементов

Часть элементов рамки таблицы (:elem) является собственно рамкой таблицы. Она определяет тип элементов рамки и ее размерность. Эта часть также определяет тип элементов таблицы и максимальный размер таблицы.

Данные из части элементов можно считать и записать путем обращения к соответствующему элементу, например, `pr:PM6:BW:elem(3)` или `pr:QJT:elem(2,6,3)` или путем обращения ко всей рамке таблицы, `pr:PM6:BW`.

Часть элементов рамки таблицы состоит из двух вложенных частей. Внешняя часть является фиксированной рамкой, которая определяется типом таблицы, а внутренняя часть является переменной таблицей, диапазон изменения которой ограничен рамкой. Рамка определяет максимальный размер и тип элементов переменной таблицы.

Рамка таблицы

Рамка таблицы - это пространство, зарезервированное для элементов таблицы. В рамке находится фиксированное число элементов фиксированного типа. Рамку можно определить посредством части элементов типа `table` следующим образом:

- `:elem(x,y,z)` ЕСТЬ `type1`

Это значит, что в блоке данных находятся элементы `x`, `y` и `z`, расположенные вдоль соответствующих направлений размерности таблицы. Тип элементов есть `type1`. Общее число элементов $x * y * z$.

Тип и размерность данных хранятся в следующих элементах заголовка:

- `:dim` (= размерность рамки = (x,y,z))
- `:elemtype` (= тип элементов рамки = `type1`)

Элементы заголовка, определенные рамкой таблицы, невозможно изменить в процессе конфигурирования или выполнения. Рамка не имеет возможности конфигурирования, а определяется типом.

Индексы рамки начинаются с 1. Индекс первого элемента есть 1 (или 1,1 или же 1,1,1). Элементов с индексами 0 (или 0,0 или 0,0,0) не существует.

Элементы рамки можно считать и записать путем обращения к требуемым элементам, например, `pr:PM6:BW:elem(3)`.

В отличие от таблицы, рамка имеет фиксированное число элементов, которое не может быть изменено путем добавления или удаления элементов.

Таблица

Таблица - это набор элементов в пространстве, зарезервированных рамкой. Максимальный размер и тип элементов таблицы определяется рамкой. Другие свойства таблицы определяются следующими членами заголовка:

- `:mode` (= режим таблицы)
- `:size` (= размер таблицы)
- `:base` (= начало таблицы в рамке)

Значения `:mode`, `:size` и `:base` могут быть изменены в процессе конфигурирования. Значения `:size` и `:base` могут быть изменены также и в процессе выполнения. Они должны, однако, находиться в пределах, определенных рамкой. Более того, между ними имеются некоторые зависимости, которые следует принять в расчет. Эти зависимости будут рассмотрены при описании каждого элемента.

Если рамка является элементом функционального блока, значения, присваиваемые по умолчанию элементам `:mode`, `:size` и `:base`, будут указаны в типе функционального блока. Разрешение на изменение этих значений на этапе конфигурирования или выполнения зависит от функционального блока.

Индексы таблицы начинаются с 1. Индекс первого элемента 1 (или 1,1 или же 1,1,1). Элементов с индексами 0 (или 0,0 или 0,0,0) не существует. Индексы таблицы расположены вдоль тех же направлений, что и индексы рамки.

Обращение к элементам таблицы (например, для чтения или записи) производится только через рамку. Команда адресации может указывать в каждый момент времени только на один элемент, например, `pr:PM6:BW:elem(4)`. Индекс табличного элемента в рамке можно вычислить при помощи параметров `:mode`, `:size` и `:base`. Например, если `:mode = array`, то:

- `:elem(:base)` = первый элемент таблицы
- `:elem(:base-1+n)` = n-ый элемент таблицы
- `:elem(:base-1+:size)` = последний элемент таблицы

Значения элементов таблицы можно считать и записать в процессе выполнения модуля. Также в процессе выполнения могут быть добавлены новые элементы, а старые удалены. Добавление или удаление возможно при помощи соответствующего функционального блока. Эти операции также можно выполнить путем добавления новых параметров `:size` и `:base` непосредственно в заголовок табличного типа. Второй метод является не совсем безопасным, поскольку пользователь сам должен проследить, не нарушены ли пределы `:size` и `:base`. Настоятельно рекомендуется по возможности использовать функциональный блок при добавлении и удалении элементов в таблицу. Другим безопасным способом добавления и удаления элементов в таблицу является корректировка содержимого табличного типа (как заголовка, так и части элементов со всеми элементами одновременно). Эту операцию можно выполнить путем связывания рамки таблицы с другой рамкой таблицы того же типа.

1.3 ТАБЛИЧНЫЙ РЕЖИМ

Таблицы могут быть разделены на основе параметра режима `:mode` на разные группы по различным признакам. Возможными значениями `:mode` являются :

- `array` (массив)
- `ring` (кольцевой массив)
- `stack` (стек - поддерживается еще не всеми функциональными блоками)
- `queue` (очередь - поддерживается еще не всеми функциональными блоками)

1.3.1 Массив

Массив, как правило, представляет собой набор параллельных элементов, созданных в одно время. Элементы размещаются в рамке таблицы один за другим без промежутков и пробелов между соседними элементами. Таблица не может быть продолжена после последнего элемента рамки, `:elem(:dim)`. Массивами являются обычный вектор, матрица и 3-мерная таблица.

В массиве индексы таблицы и рамки следующие:

- `:elem(1)` = первый элемент рамки
- `:elem(:base)` = первый элемент таблицы
- `:elem(:base-1+n)` = n-ый элемент таблицы
- `:elem(:base-1+:size)` = последний элемент таблицы
- `:elem(:dim)` = последний элемент рамки

Если `:base = (1,1,1)` как в большинстве массивов, индексы в таблице и рамке будут такими же. Следовательно, адресацию к таблице можно осуществить через рамку без дополнительных вычислений.

В массиве добавление или удаление элементов обычно означает изменение размера `:size` при неизменной базе `:base`.

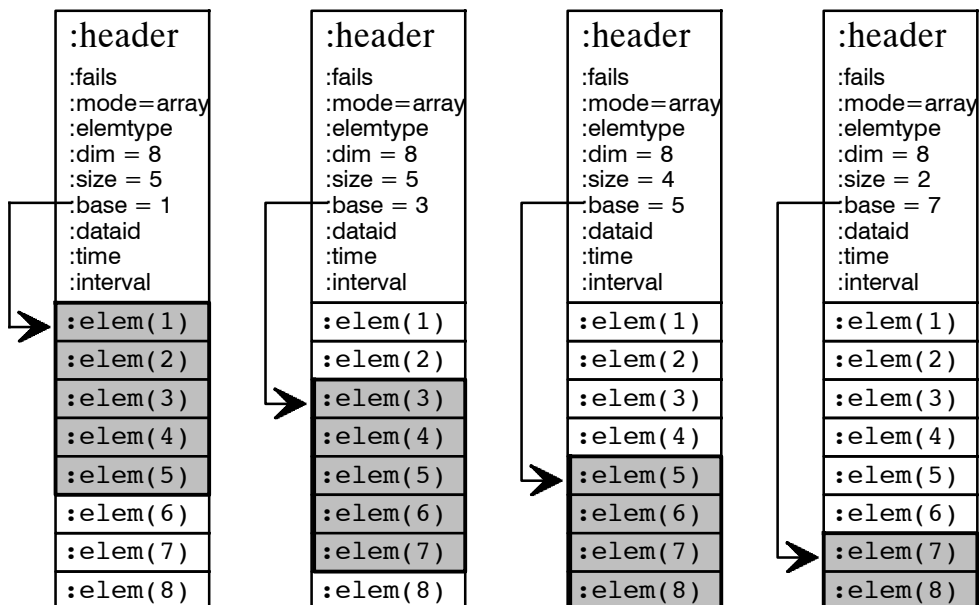


Рисунок 2 Одномерные массивы

:header							
:fails							
:mode=array							
:elemtype							
:dim = 8,8							
:size = 4,6							
:base = 1,1							
:dataid							
:time							
:interval							
→ :e(1,1)	:e(1,2)	:e(1,3)	:e(1,4)	:e(1,5)	:e(1,6)	:e(1,7)	:e(1,8)
:e(2,1)	:e(2,2)	:e(2,3)	:e(2,4)	:e(2,5)	:e(2,6)	:e(2,7)	:e(2,8)
:e(3,1)	:e(3,2)	:e(3,3)	:e(3,4)	:e(3,5)	:e(3,6)	:e(3,7)	:e(3,8)
:e(4,1)	:e(4,2)	:e(4,3)	:e(4,4)	:e(4,5)	:e(4,6)	:e(4,7)	:e(4,8)
:e(5,1)	:e(5,2)	:e(5,3)	:e(5,4)	:e(5,5)	:e(5,6)	:e(5,7)	:e(5,8)
:e(6,1)	:e(6,2)	:e(6,3)	:e(6,4)	:e(6,5)	:e(6,6)	:e(6,7)	:e(6,8)
:e(7,1)	:e(7,2)	:e(7,3)	:e(7,4)	:e(7,5)	:e(7,6)	:e(7,7)	:e(7,8)
:e(8,1)	:e(8,2)	:e(8,3)	:e(8,4)	:e(8,5)	:e(8,6)	:e(8,7)	:e(8,8)

:header							
:fails							
:mode=array							
:elemtype							
:dim = 8,8							
:size = 5,5							
:base = 3,3							
:dataid							
:time							
:interval							
:e(1,1)	:e(1,2)	:e(1,3)	:e(1,4)	:e(1,5)	:e(1,6)	:e(1,7)	:e(1,8)
:e(2,1)	:e(2,2)	:e(2,3)	:e(2,4)	:e(2,5)	:e(2,6)	:e(2,7)	:e(2,8)
:e(3,1)	→ :e(3,2)	:e(3,3)	:e(3,4)	:e(3,5)	:e(3,6)	:e(3,7)	:e(3,8)
:e(4,1)	:e(4,2)	:e(4,3)	:e(4,4)	:e(4,5)	:e(4,6)	:e(4,7)	:e(4,8)
:e(5,1)	:e(5,2)	:e(5,3)	:e(5,4)	:e(5,5)	:e(5,6)	:e(5,7)	:e(5,8)
:e(6,1)	:e(6,2)	:e(6,3)	:e(6,4)	:e(6,5)	:e(6,6)	:e(6,7)	:e(6,8)
:e(7,1)	:e(7,2)	:e(7,3)	:e(7,4)	:e(7,5)	:e(7,6)	:e(7,7)	:e(7,8)
:e(8,1)	:e(8,2)	:e(8,3)	:e(8,4)	:e(8,5)	:e(8,6)	:e(8,7)	:e(8,8)

Рисунок 3 Двумерные массивы

1.3.2 Кольцевой массив

Кольцевой массив представляет собой таблицу, концы рамки которой связаны между собой. Поэтому кольцевой массив можно продолжить после последнего элемента рамки `:elem(:dim)` элементом `:elem(1)`. Элементы хранятся в рамке последовательно без промежутков. Кольцевой массив используют, например, при задержках и в трендах. В кольцевом массиве индексы таблицы и рамки имеют следующий вид :

- `:elem(1)` = первый элемент рамки
- `:elem(:base)` = первый элемент кольцевого массива
- `:elem((:base-1+n) modulo :dim)` = n-ый элемент кольцевого массива
- `:elem((:base-1+:size) modulo :dim)` = последний элемент кольцевого массива
- `:elem(:dim)` = последний элемент рамки

Запись `((:base-1+n) modulo :dim)` означает, что элементы кольцевого массива продолжаются после последнего элемента рамки `:elem(:dim)` первым элементом рамки `:elem(1)`.

Обычно кольцевой массив используют с базой `:base /= (1,1,1)`. Индексы кольцевого массива отличаются от индексов рамки. Это следует учитывать при адресации элементов кольцевого массива через элементы рамки.

Новый элемент может быть добавлен в начало кольцевого массива. Самый старый элемент в конце кольцевого массива должен быть также одновременно удален. Добавление или удаление обычно производят при помощи функционального блока. Индекс элемента, подлежащего добавлению, в рамке равен `:base-1`, а индекс элемента, подлежащего удалению, равен `((:base-1+:size) modulo :dim)`. После добавления и удаления функциональный блок устанавливает `:base = :base-1`, а параметр `:size` оставляет неизменным. Следовательно, длина кольцевого массива не меняется. Вместо этого, кольцевой массив вращается в обратном направлении вокруг рамки, и рамка никогда не переполнится.

Наиболее общим применением кольцевого массива являются тренды (см. раздел "Функциональные блоки" глава 2 "ahs2 - функциональный блок аналоговой предыстории").

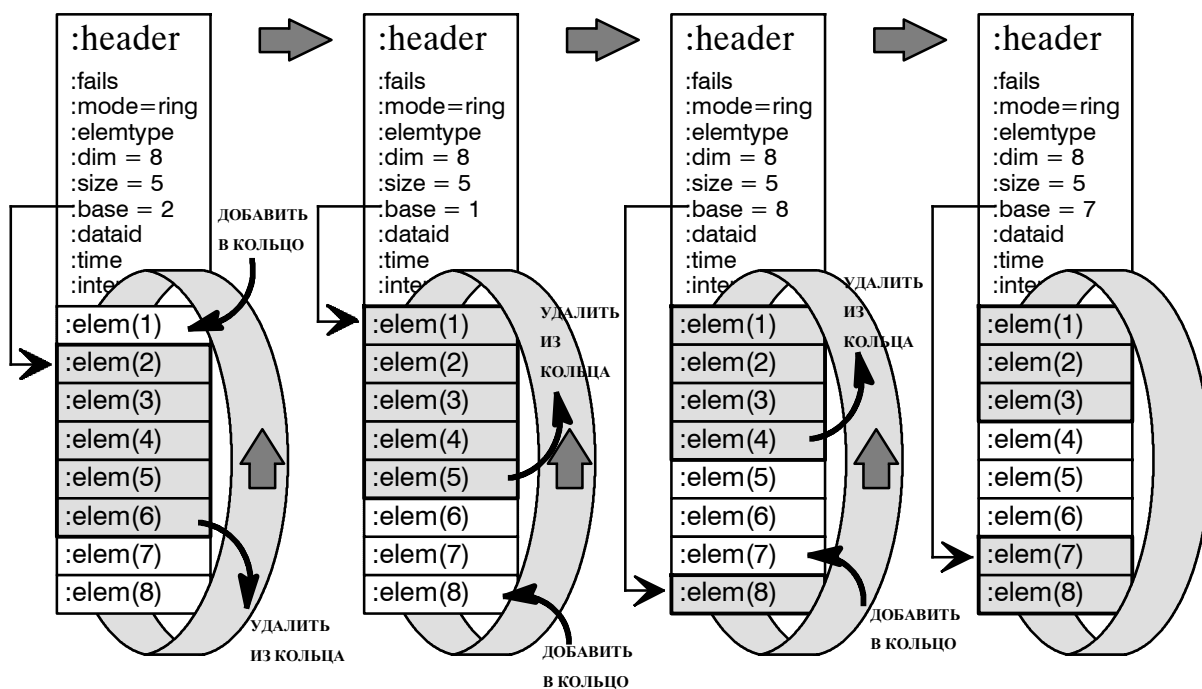


Рисунок 4 Операции добавления и удаления в одномерном кольцевом массиве

1.3.3 Стек (поддерживается еще не всеми функциональными блоками)

Стек представляет собой массив, в котором новый элемент может быть добавлен на вершину стека и в котором последний добавленный элемент может быть удален из вершины стека. Добавление или удаление обычно осуществляют при помощи функционального блока. Индекс элемента, подлежащего добавлению, в рамке равен `:base-1`, а индекс элемента, подлежащего удалению, в рамке равен `:base`. После добавления происходит установка `:base = :base-1` и `:size = size+1`. После удаления происходит установка `:base = :base+1` и `:size = size-1`. Стек заполнен, если `:size = :dim`. Стек пуст, если `:size = 0`. В заполненный стек новый элемент добавить невозможно, аналогично нельзя удалить элемент из пустого стека.

Обычно стеки используют при `:base /= (1,1,1)`. Индексы стека отличаются от индексов рамки. Это следует учитывать при адресации элементов стека через элементы рамки.

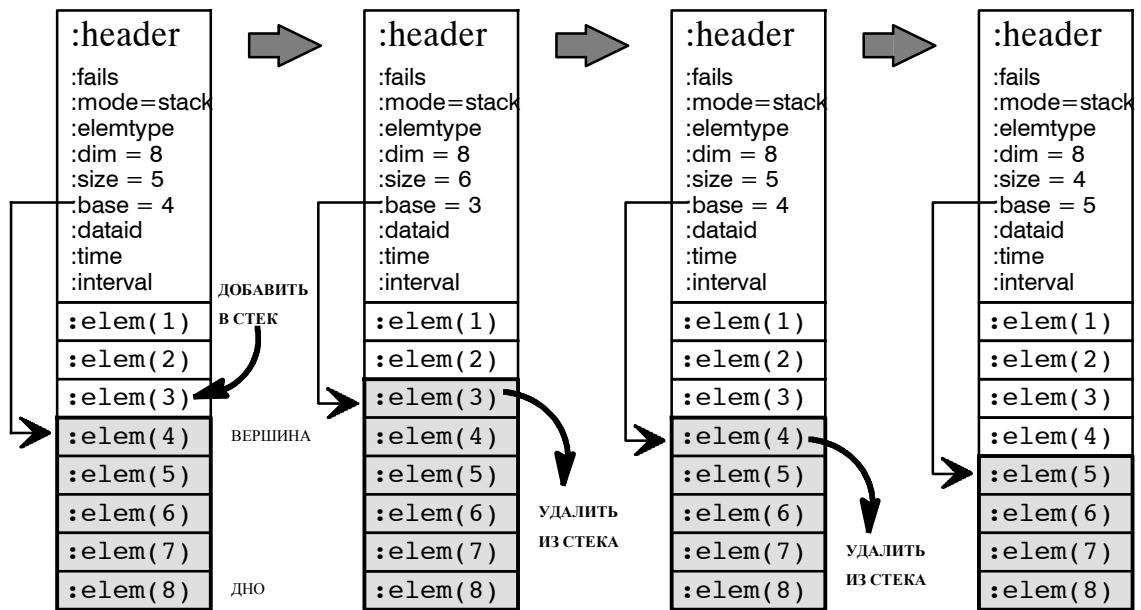


Рисунок 5 Операции добавления и удаления в одномерном стеке

1.3.4 Очередь (поддерживается еще не всеми функциональными блоками)

Очередь представляет собой кольцевой массив, в котором новый элемент может быть добавлен в конец очереди и в котором самый старый элемент может быть удален из начала очереди. Добавление или удаление обычно осуществляют при помощи функционального блока. Индекс элемента, подлежащего добавлению, в рамке равен $((\text{base} + \text{size}) \bmod \text{dim})$, а индекс элемента, подлежащего удалению, в рамке равен base . После добавления происходит установка $\text{size} = \text{size} + 1$. После удаления происходит установка $\text{base} = \text{base} + 1$ и $\text{size} = \text{size} - 1$. Очередь заполнена, если $\text{size} = \text{dim}$. Очередь пуста, если $\text{size} = 0$. Если очередь полна, то новый элемент не может быть в нее добавлен, также же нельзя удалить элемент из пустой очереди.

Обычно очереди используют при $\text{base} \neq (1, 1, 1)$. Следовательно, индексы очереди отличаются от индексов рамки. Это следует учитывать при обращении к элементам очереди через элементы рамки.

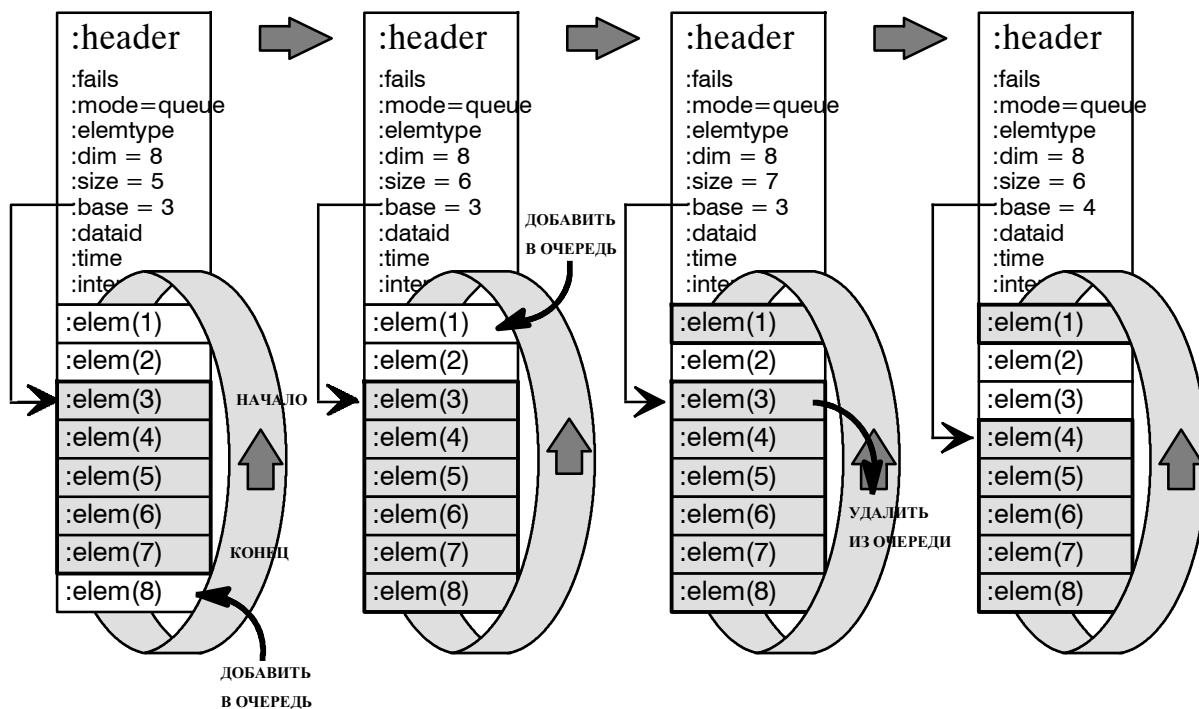


Рисунок 6 Операции добавления и удаления в одномерной очереди

1.4 АДРЕСАЦИЯ ВНЕШНЕЙ ТАБЛИЦЫ

Данные внешней рамки таблицы можно считать и записать путем обращения к требуемому элементу заголовка, например, `pr:PM6:BW:header.size(3)`, ко всему заголовку, например, `pr:PM6:BW:header`, к требуемому элементу рамки или таблицы, например, `pr:PM6:BW:elem(4)` или ко всей рамке таблицы, например, `pr:PM6:BW`. При адресации внешней рамки таблицы необходимо учитывать нижеизложенное.

1.4.1 Передача данных

Формирование в модуле внешней точки данных, например, `pr:PM6:BW1`, не инициирует передачу данных. Только посредством связи объекта в модуле с внешней точкой данных или с ее частью будет инициирована передача данных. Будет передана только та часть внешних данных, которая связана с объектом в модуле, например, будет передан `pr:PM6:BW:header`. Объект в модуле может быть портом или элементом функционального блока. В обоих случаях внешние таблица и скалярный тип функционируют одинаково.

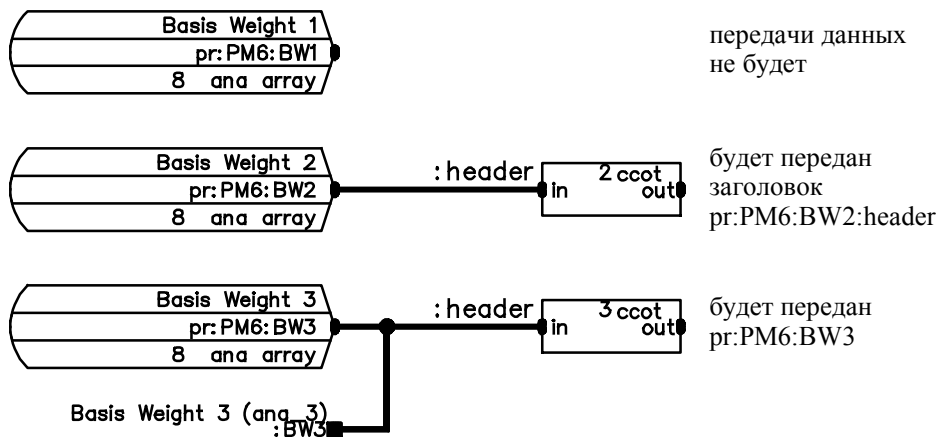


Рисунок 7 Передача внешних данных в модуль

1.4.2 Первый элемент

Если в модуле задан внешний табличный тип `external`, например, `pr:PM6:BW` представлен в модуле, и объект (порт или элемент функционального блока) в том же самом модуле связан с его первым элементом с помощью указателя `:elem(1)`, то:

- на стадии загрузки модуля в регистратор связи будет возвращено предупреждение о несовместимости типа,
- данные, т.е. `:elem(1)`, не будут переданы в модуль, а связь будет помечена как невыполненная,
- в PCS никакие другие элементы внешней таблицы переданы не будут, несмотря на то, что в модуле объекты были связаны с ними указателями `:elem(2)`, `:elem(3)`...

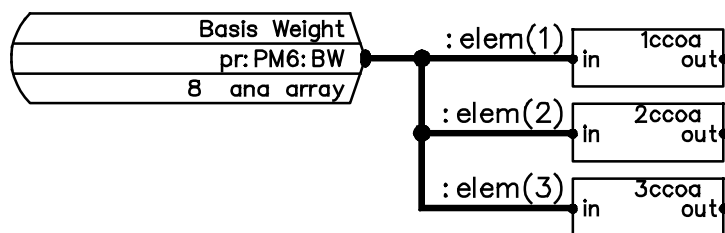


Рисунок 8 Неправильное обращение к первому элементу внешней таблицы

Во избежание вышеуказанной проблемы следует использовать один из рассмотренных ниже способов связи объекта модуля с первым элементом внешней таблицы.

1. Следует указать дополнительный интерфейсный порт любого типа "any" (обратите внимание! не табличного типа table), например, :BW1, тип которого определен как тип таблицы table, например, ana_3, в диалоговом окне инструментов CAD. Этот способ полезен в случае, когда почти все элементы внешней таблицы необходимы в модуле.

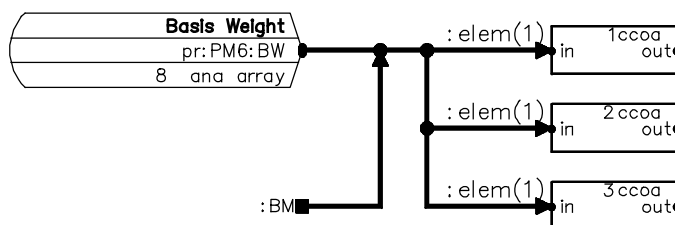


Рисунок 9 Правильное обращение к первому элементу внешней таблицы

2. Каждый требуемый элемент внешней таблицы, например, pr:PM6:BW2, будет представлен как отдельная внешняя точка данных скалярного типа, например, pr:PM6:BW2:elem(1), pr:PM6:BW2:elem(2)... и объекты в модуле будут связаны с ними. Этот способ полезен в случае, когда лишь несколько элементов внешней таблицы необходимы в модуле.

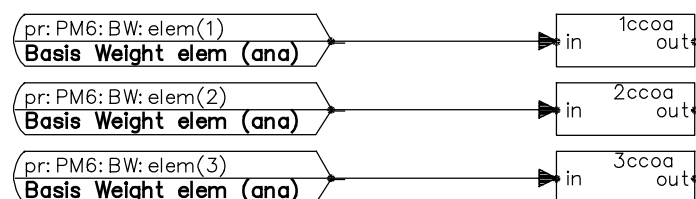


Рисунок 10 Правильное обращение к первому элементу внешней таблицы

3. Первый элемент внешней таблицы, например, pr:PM6:BW3, будет представлен отдельная внешняя точка данных скалярного типа, например, pr:PM6:BW3:elem(1). Для других элементов, необходимых в модуле, вся таблица будет представлена как внешняя точка данных табличного типа, например, pr:PM6:BW3. Объекты, запрашивающие первый элемент, будут связаны со скалярным внешним типом external, например, pr:PM6:BW3:elem(1), без каких-либо дополнительных указателей. Объекты, которым необходимы другие элементы, будут связаны с внешним табличным типом external, например, pr:PM6:BW3, соответствующими указателями :elem(2), :elem(3)...

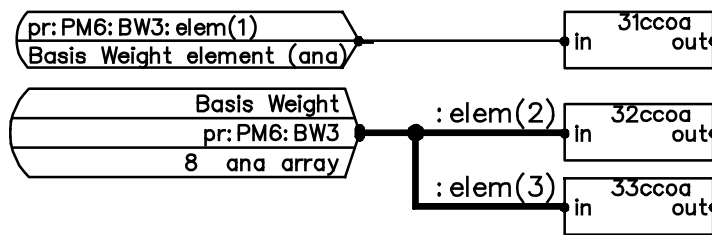


Рисунок 11 Правильное обращение к первому элементу внешней таблицы

1.4.3 Функционирование

Если тип таблицы внешний external, например, pr:PM6:BW, задан в модуле в OPS, а скалярные элементы функционального блока в модуле связаны с внешними элементами указателями :elem(1), :elem(2)... , то функциональные блоки могут отображать значения элементов (кроме :elem(1), см. выше), но над ними невозможно производить какие-либо действия. Если элементы должны быть работоспособными, каждый элемент внешней таблицы следует задать как отдельную, скалярную внешнюю точку данных (например, pr:PM6:BW:elem(1), pr:PM6:BW:elem(2),...), а функциональные блоки в модуле должны быть связаны с этими скалярными точками данных. Например, в рассмотренном ниже модуле элементы внешней таблицы отображаются на дисплее и доступны).

REPRESENTATION_PART

EXTERNALS

```
pr:PM6:BW:elem(1) TYPE ana TRANSFER 192,10,0,0;
pr:PM6:BW:elem(2) TYPE ana TRANSFER 192,10,0,0;
pr:PM6:BW:elem(3) TYPE ana TRANSFER 192,10,0,0;
```

FUNCTIONAL_PART

```
1anan
def<(28,566,22,0,...)
pos!<-
pdata<pr:PM6:BW:elem(1)
pdataop<pr:PM6:BW:elem(1)
alm<-
;
```

```
2anan
def<(60,566,22,0,...)
pos!<-
pdata<pr:PM6:BW:elem(2)
pdataop<pr:PM6:BW:elem(2)
alm<-
;
```

```
3anan
def<(92,566,22,0,...)
pos!<-
pdata<pr:PM6:BW:elem(3)
pdataop<pr:PM6:BW:elem(3)
alm<-
;
```

END

1.5 ЭЛЕМЕНТЫ ЗАГОЛОВКА

fails

Тип:	fails
Значение по умолчанию:	48
Описание:	Биты неисправности

Элемент `:fails` содержит общие биты ошибок всех используемых элементов таблицы (элементы размера `size`). Он суммирует достоверность табличных элементов, как видно источнику (обычно функциональному блоку). Он ничего не сообщает о битах ошибки отдельных элементов. Каждый функциональный блок может иметь свою логику установки параметра `:fails`. Если иное не определено, `:fails` представляет собой логически объединенные по "ИЛИ" биты ошибок всех используемых элементов таблицы (элементы размера `size`).

Такую же логику используют при установке элемента `:fails` независимо от того, имеют ли элементы таблицы биты ошибок. Если они не имеют битов ошибок, `:fails` будет установлен таким образом, как если бы они имели таковые.

Если элементы таблицы не имеют битов ошибок, параметр `:header:fails` является единственным способом проверить достоверность элементов.

Биты ошибок других элементов заголовка (например, `:dataid`, `:time` и `:interval`) не оказывают влияния на `:header:fails`. Эти элементы независимы от элементов таблицы и имеют собственные биты ошибок. При необходимости их можно считать отдельно.

mode(16)

Тип:	char
Значение по умолчанию:	array
Описание:	Режим таблицы ()

Элемент `:mode` содержит структурные данные таблицы. Он сообщает о режиме таблицы. Режим в основном указывает поведение таблицы (`:base` и `:size`) при добавлении в нее новых элементов или удалении старых.

Элемент `:mode` представляет собой 16-символьный текст (15 символов + `NULL`). Возможные значения `:mode` следующие:

- массив
- кольцевой массив
- (- очередь)
- (- стек)

Обычно функциональный блок необходим для того, чтобы поведение таблицы было таким, как указывает элемент `:mode`. Например, `ahs2` может обрабатывать таблицы, у которых `:mode = ring` (кольцевой массив).

Если указан режим иного типа, кроме таблиц (`table`), он должен быть скалярным `"scalar"`.

При конфигурировании таблицы с помощью символа таблицы или типа символа `"any"` режим `:mode` можно изменить в окне заполнения формы и в диалоговом прямоугольнике. Однако существуют взаимные зависимости между `:mode`, `:dim`, `:size` и `:base`, которые следует учитывать. См. `:size` и `:base`. Во время исполнения `:mode` изменять запрещается.

elemtype(8)

Тип:	char
Значение по умолчанию:	ana
Описание:	Тип элемента ()

Элемент :elemtype содержит данные структуры рамки. Он указывает имя типа элементов рамки (и таблицы) как текст. Таким образом, значение :elemtype то же самое, что и тип элементов рамки (и элементов таблицы), определенных членом :elem типа таблицы. Значение :elemtype также отображается как аббревиатура имени типа таблицы, например, ana_3, bin_4 и т.д.

Когда конфигурирование таблицы использует символ таблицы, :elemtype может быть изменен в окне заполнения формы, но не в окне диалога. Окно диалога создано только для типа элемента, определенного в окне заполнения формы. Если символ типа "anu" используют при конфигурировании таблицы, имя типа таблицы, указанное в диалоговом окне, и начальное значение, присвоенное :elemtype, должны соответствовать одно другому. Во время исполнения запрещается изменять параметр :elemtype.

dim(3)

Тип:	intl
Значение по умолчанию:	48 1 48 1 48 1
Описание:	Размеры рамки ()

Член :dim содержит данные структуры рамки. Он указывает размеры рамки и максимальный фактический размер (:size) таблицы. Значение :dim должно быть то же самое, как размеры рамки, заданные в члене :elem типа таблицы. Значение :dim также видно в имени типа таблицы как код (как показатель степени двух), например, ana_3, bin_4 и т.д.

Когда конфигурирование таблицы использует символ таблицы, :dim может быть изменен в окне заполнения формы и в окне диалога. Когда символ типа "любой" используют в конфигурировании таблицы, имя типа таблицы, заданное в окне диалога, и начальное значение, заданное для :dim, должны соответствовать друг другу. Во время исполнения время :dim не должно быть изменено.

Если в :dim установлены какие-нибудь биты ошибки иные, чем ovf, dis, old или seh, функциональные блоки не будут обрабатывать таблицу, но укажут ошибку структуры рамки.

size(3)

Тип:	intl
Значение по умолчанию:	48 1 48 1 48 1
Описание:	Размер таблицы ()

Элемент :size содержит данные структуры таблицы. Он указывает (фактический) размер таблицы. Это число элементов, зарезервированных для таблицы в рамке (начиная с :base) в направлениях размерностей. Первый элемент таблицы в рамке :elem(:base), например, для "array" последний будет :elem(:base-1+:size). Если :size = 0, в таблице нет элементов.

Разрешенный диапазон :size во время конфигурирования следующий:

- (0,0,0) =< :size =< :dim
- :base-1+:size =< :dim

Разрешенный диапазон :size во время выполнения следующий:

- (0,0,0) =< :size =< :dim (для всех режимов :mode)
- :base-1+:size =< :dim (когда :mode = array или stack)

Во время конфигурирования :size имеет более жесткие ограничения, чем во время выполнения. Это связано с тем, что во время конфигурирования все таблицы с разным :mode будут обработаны как таблицы типа "array", чтобы сделать более легкой подачу начального значения. Таким образом, во время конфигурирования таблицы не могут быть обработаны как кольцевые массивы rings. Только во время выполнения таблицы в режиме "ring" (в отличие от array и stack) будут освобождены для вращения вокруг рамки.

Значение :size по умолчанию в типах таблиц и в формах заполнения символов таблицы есть size = dim, без битов ошибки. Значение :size может быть изменено на стадии конфигурирования и во время выполнения.

Если какие-нибудь биты ошибки, отличные от ovf, dis, old или sex, установлены в :size, функциональные блоки не будут обрабатывать таблицу, но укажут ошибку структуры таблицы.

base(3)

Тип: intl
Значение по умолчанию: 48 1 48 1 48 1
Описание: Базовая точка таблицы ()

Элемент :base содержит данные структуры таблицы. Он указывает индекс первого элемента таблицы в рамке. Таким образом, первый элемент таблицы в рамке :elem (:base), например, для "array" последний будет :elem(:base-1+:size).

Разрешенный диапазон base следующий:

- (1,1,1) =< base =< dim

Значение :base по умолчанию в типах таблиц и в формах заполнения символов таблицы есть первый элемент в рамке (1,1,1), без битов ошибки. Значение :base может быть изменено на стадии конфигурирования и во время выполнения.

Если какие-нибудь биты ошибки, отличные от ovf, dis, old или sex, установлены в :size, функциональные блоки не будут обрабатывать таблицу, но укажут ошибку структуры таблицы.

dataid

Тип: intl
Значение по умолчанию: 48 0
Описание: Данные идентификации ()

Элемент :dataid - это идентификационный номер данных таблицы. Он указывает индивидуальное значение таблицы среди расположенных в хронологическом порядке значений той же таблицы. Член :dataid может быть использован, например, следующим образом:

- Член :dataid указывает исходный номер единственного значения таблицы в ряду последовательных значений.
- Член :dataid указывает изменение в значении таблицы. В этом случае значение :dataid будет увеличиваться или уменьшаться на единицу каждый раз при изменении значения таблицы.
- Член :dataid связывает значение таблицы с другими данными, которые имеют тот же самый параметр :dataid.

Точный смысл :dataid зависит от функционального блока, который его устанавливает. Если не задано иное, :dataid указывает изменение в значении таблицы с помощью приращения значения входа :dataid.

time

Тип:	binev
Значение по умолчанию:	48 0 0 0 0 0 5 1 1 136 14
Описание:	Время ()

Член :time - это метка времени таблицы. Ее тип - binev. Ее элемент :time:bin-time, который является типом времени, указывающим время последнего обновления таблицы. Ее элемент :time:binstat, тип которого bin, изменяет состояние каждый раз при обновлении :time:bin-time. Обычно в то же время обновляют :dataid.

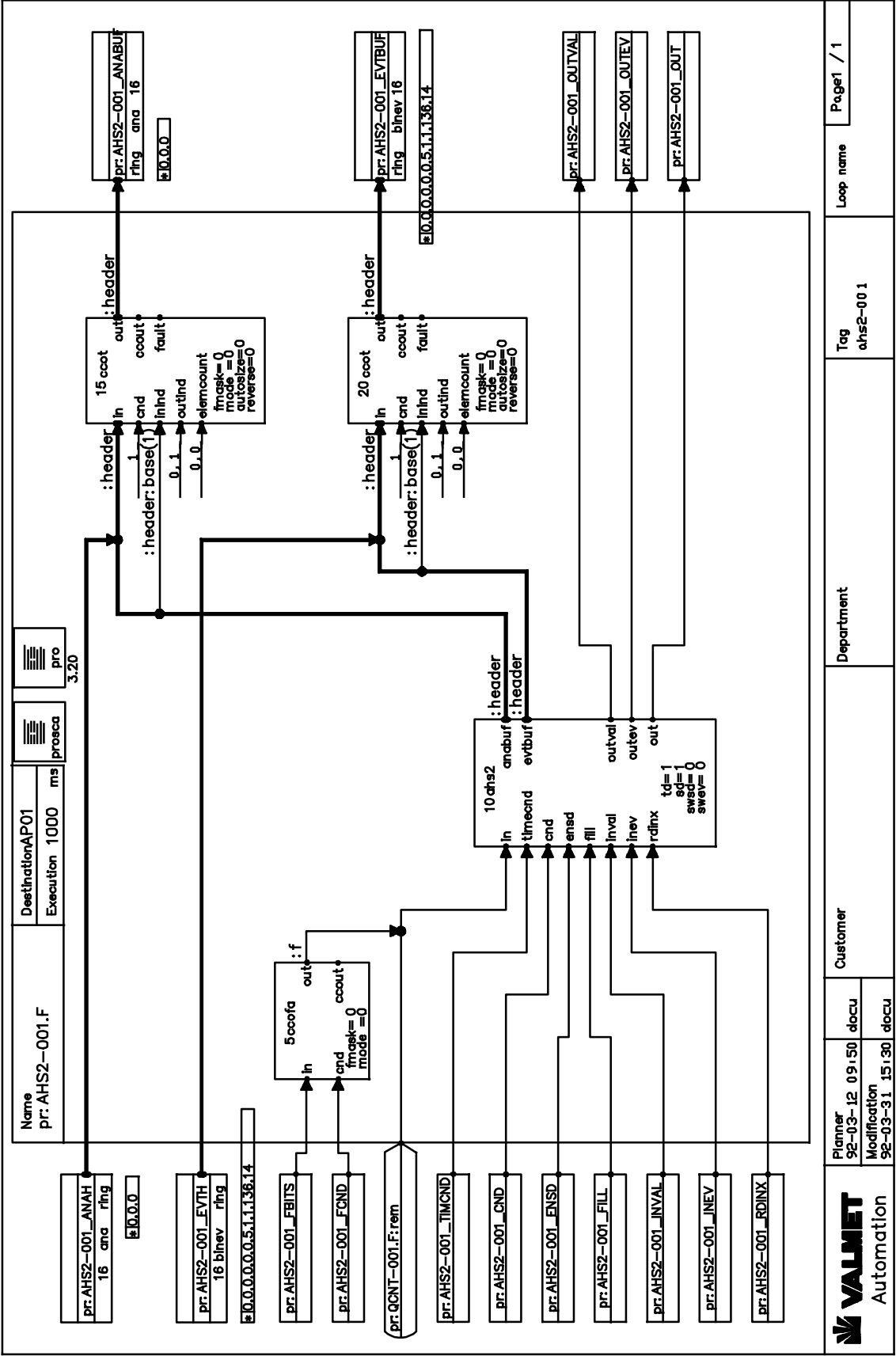
interval

Тип:	ana
Значение по умолчанию:	48 0.0
Описание:	Интервал элементов ()

Член :interval указывает промежуток между последовательными элементами (строками или столбцами) таблицы, если элементы расположены равномерно. Единица измерения интервала :interval для каждого случая будет задана отдельно (в функциональных блоках). Если элементы таблицы расположены неравномерно или не имеют ничего общего один с другим, :interval = 0.

В трендах, основанных на времени, :interval (>0) указывает время в секундах между последовательными элементами тренда. Если в тренде :interval = 0, тренд основан на событии.

1.6 ПРИМЕР КОНФИГУРАЦИИ 1




```

0,0,0,
0,0,0);
pr:AHS2-001_TIMCND TYPE txt10 < ("*****00");

INTERFACE
  MODSTAT TYPE ktstat < (1,1,0);

FUNCTIONAL_PART

5ccofa
out> pr:QCNT-001.F:rem:f
cnd< pr:AHS2-001_FCND
in< pr:AHS2-001_FBITS
fmask= (0)
mode= (0)
;

10ahs2
in< pr:QCNT-001.F:rem
evtbuf> pr:AHS2-001_EVTH:header
anabuf> pr:AHS2-001_ANAH:header
rdinx< pr:AHS2-001_RDINX
inev< pr:AHS2-001_INEV
inval< pr:AHS2-001_INVAL
fill< pr:AHS2-001_FILL
ensd< pr:AHS2-001_ENSD
cnd< pr:AHS2-001_CND
timecnd< pr:AHS2-001_TIMCND
out> pr:AHS2-001_OUT
outev> pr:AHS2-001_OUTEV
outval> pr:AHS2-001_OUTVAL
td= (1)
sd= (1)
swsd= (0)
swev= (0)
swct= (0)
swti= (0)
cont= (0)
fbmask= (255)
;

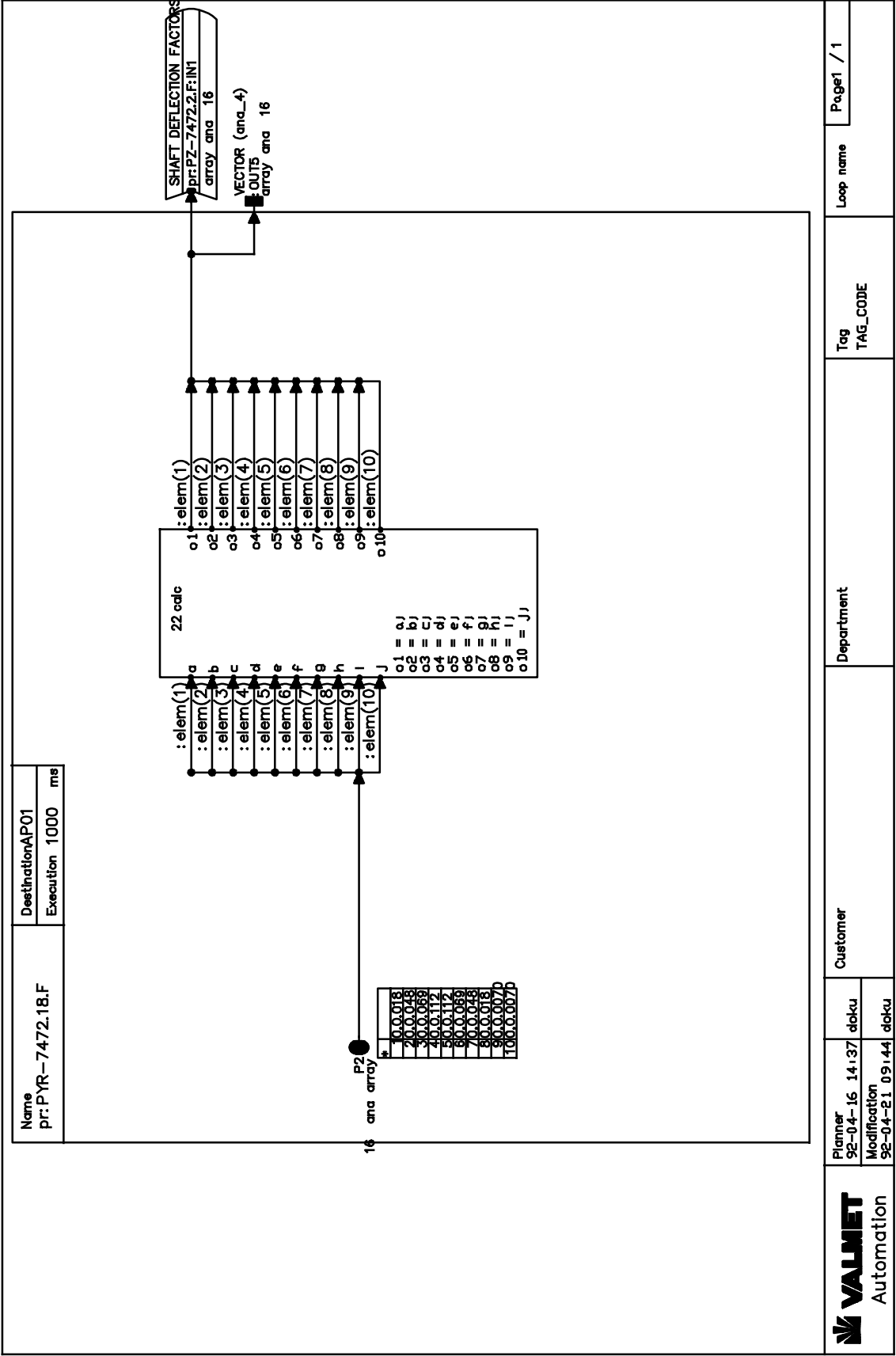
15ccot
in< pr:AHS2-001_ANAH:header
inind< pr:AHS2-001_ANAH:header:base(1)
cnd< (1)
outind< (0,1)
elemcount< (0,0)
out> pr:AHS2-001_ANABUF:header
fmask= (0)
mode= (0)
autosize= (0)
reverse= (0)
;

20ccot
in< pr:AHS2-001_EVTH:header
inind< pr:AHS2-001_EVTH:header:base(1)
cnd< (1)
outind< (0,1)
elemcount< (0,0)
out> pr:AHS2-001_EVTBUF:header
fmask= (0)
mode= (0)
autosize= (0)
reverse= (0)
;

END

```

1.7 ПРИМЕР КОНФИГУРАЦИИ 2



ADMINISTRATION_PART

NAME: pr:PZR-7472.18.F
 TYPE: function
 STATUS: incomplete
 CREATOR: doku
 CREATED: 92-04-16 14:37
 MODIFIER: fbcad
 MODIFIED: 92-04-16 14:48
 DESTINATION: AP01
 EXECUTION: 1000
 ORDINAL: 20
 DESCRIPTION:

REPRESENTATION_PART

EXTERNALS

pr:PZR-7472.2.F:IN1 TYPE ana_4 TRANSFER 65,10,0,0 "SHAFT DEFLECTION FACTORS";

LOCALS

P2(1) TYPE ana_4=(
 0,"array","ana",
 0,16,0,1,0,1,0,10,0,1,0,1,
 0,1,0,1,0,1,
 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
 0,0,0,18,0,0,0,48,0,0,0,69,0,0,1,12,
 0,0,1,12,0,0,0,69,0,0,0,48,0,0,0,18,
 0,0,0,0,70,0,0,0,0,70) "INITIAL VALUES";

INTERFACE

OUT5 TYPE ana_4 "VECTOR (ana_4)" < pr:PZR-7472.2.F:IN1 ;
 MODSTAT TYPE ktstat < (1,1,0);

FUNCTIONAL_PART

CALCULATE 22calc

CONNECT

o1 TYPE ana > pr:PZR-7472.2.F:IN1:elem(1);
 o2 TYPE ana > pr:PZR-7472.2.F:IN1:elem(2);
 o3 TYPE ana > pr:PZR-7472.2.F:IN1:elem(3);
 o4 TYPE ana > pr:PZR-7472.2.F:IN1:elem(4);
 o5 TYPE ana > pr:PZR-7472.2.F:IN1:elem(5);
 o6 TYPE ana > pr:PZR-7472.2.F:IN1:elem(6);
 o7 TYPE ana > pr:PZR-7472.2.F:IN1:elem(7);
 o8 TYPE ana > pr:PZR-7472.2.F:IN1:elem(8);
 o10 TYPE ana > pr:PZR-7472.2.F:IN1:elem(10);
 o9 TYPE ana > pr:PZR-7472.2.F:IN1:elem(9);
 j TYPE ana < P2:elem(10);
 i TYPE ana < P2:elem(9);
 h TYPE ana < P2:elem(8);
 g TYPE ana < P2:elem(7);
 f TYPE ana < P2:elem(6);
 e TYPE ana < P2:elem(5);
 d TYPE ana < P2:elem(4);
 c TYPE ana < P2:elem(3);
 b TYPE ana < P2:elem(2);
 a TYPE ana < P2:elem(1);

FORMULAS

o1 = a;
 o2 = b;
 o3 = c;
 o4 = d;
 o5 = e;
 o6 = f;
 o7 = g;

```
o8 = h;  
o9 = i;  
o10 = j;  
STOP 22calc
```

```
END
```

1.8 ccot - ФУНКЦИОНАЛЬНЫЙ БЛОК УСЛОВНОГО КОПИРОВАНИЯ ТАБЛИЦЫ

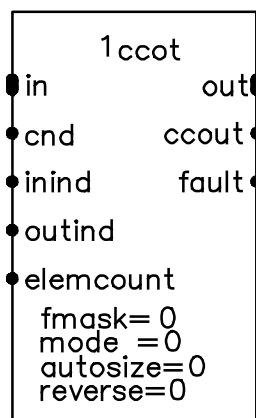
Этот функциональный блок выполняет условное копирование таблиц 1- и 2-мерного массива и кольцевых таблиц любых типов элементов и любых размерностей. Задаваемые произвольно биты ошибки могут быть маскированы в поле заголовка :fails и из каждого элемента таблицы (если он имеет поле бита ошибки), или же они могут быть скопированы без изменений.

Входная и выходная таблицы (источник и адресат) могут быть 1- или 2-мерными и независимыми один от другого. Из одномерной таблицы один или несколько элементов можно скопировать в элементы одномерной таблицы или в одну строку двумерной таблицы. Из двумерной таблицы одну или несколько строк можно скопировать в строку (строки) двумерной таблицы. Из двумерной таблицы одну строку можно скопировать в элементы одномерной таблицы.

Таблицы связаны с блоком через их элемент заголовка :header. Таблица может быть скопирована в обратном порядке путем установки параметра :reverse. Параметр :size выходной таблицы может быть сохранен неизменным после исполнения блока установкой параметра :autosize на ноль.

Далее термин "объект данных" ("data entity") соответствует минимальной копируемой единице информации. В одномерной таблице объект данных - это один элемент таблицы. В двумерной таблице - это одна строка таблицы. Элементы :elemcount, :inind и :outind всегда относятся к объекту данных таблицы.

1.8.1 Символьное представление



1.8.2 Функциональное описание

Условия копирования

ВНИМАНИЕ!

Этот функциональный блок не поддерживает копирование между двумя модулями, как другие функциональные блоки копирования. Он всегда копирует свою входную таблицу только в свою выходную таблицу. Передачу данных между модулями следует выполнить с помощью связи через внешние соединения. Если модули в том же самом рабочем цикле, условие копирования может быть соединено с интерфейсным портом модуля :MODSTAT.

Режимы выполнения 0-4 подобны тем, которые применяют для блоков скалярного условного копирования. Кроме того, имеются режимы 6, 7, 8 и 9. Режим 6 относится к изменению элементов входной таблицы :dataid и/или :size (1). Дополнительные режимы выполнения, не реализованные в соответствующих блоках скалярного условного копирования, являются комбинациями режима 6 с каждым из трех режимов переключения.

ВНИМАНИЕ!

Этот функциональный блок использует абсолютные индексы. Элементы `:inind` и `:outind` дают абсолютное расположение объектов данных от начала рамки, то есть от `:elem(1)`. Таким образом, функциональный блок предполагает, что `:base = 1,1,1`. Таблицы режимов `agtau` и `ging` обрабатываются таким же образом.

Проверка перед копированием

Если условие копирования `:cnd` есть истина (`true`) в соответствии со значением параметра режима `:mode`, то блок сначала проверит ошибки, как описано ниже. После этого он проверит, что типы элементов в таблице выхода совпадают с типами в таблице входа. Он также проверяет, чтобы таблицы были одно- или двумерные.

Если функциональный блок обнаруживает любую из указанных выше ошибок, он обновляет выход `:fault` и устанавливает биты ошибки `inv` и `der` во всех полях битов ошибки элементов и в поле `:header:fails` таблицы выхода.

Перед выполнением в нормальном рабочем цикле (когда условия выполнения - Истина (`true`), а ошибки конфигурации отсутствуют), функциональный блок проверит следующие ошибки:

- Ошибка размеров используемых таблиц входов:
В любом из трех полей `:size` используемой таблицы входа (`:header:size(1)`, `:header:size(2)` или `:header:size(3)`) установлены некоторые биты ошибки.
- Размеры таблицы входа больше, чем размерность:
Любое из трех полей `:size` используемой таблицы входа больше, чем соответствующее поле `:dim` или больше, чем результирующий выход `:dim`.

Если функциональный блок обнаружит любую из ошибок, указанных выше, тогда действия следующие:

- Ошибка в размерах используемых входных таблиц:
Если в любом из трех полей `:size` используемой таблицы входа установлены биты ошибки, тогда функциональный блок укажет ошибку с помощью обновления выхода `:fault`. Фактическая функция не выполняется.
- Размер таблицы входа больше, чем размерность:
Если любое из трех полей `:size` таблицы входа больше, чем соответствующее поле `:dim` того же члена, то функциональный блок укажет ошибку с помощью обновления выхода `:fault` и произведет выполнение с использованием значения поля `:dim` вместо неправильного поля `:size`. Если любое из трех полей `:size` таблицы входа больше, чем соответствующее поле `:dim` результирующего члена выхода, то функциональный блок укажет ошибку обновлением выхода `:fault` и помещением бита ошибки `inv` (недействительные данные) и `der` (производные данные) во все поля бита ошибки таблицы выхода и `:header:fails`. Фактическая функция не выполняется.

Копирование из входной таблицы

Если ошибок нет, блок считывает объекты данных `:elemcount` из своей таблицы `:in`, начиная с `:inind`, и копирует их в свою таблицу `:out`, начиная с `:outind` (биты ошибки маскируются с помощью `:fmask`), см. Рисунок 12. При значении по умолчанию `:elemcount(0)` все объекты данных используемой входной таблицы копируются в таблицу выхода (`in:header:size(1)`). Если параметр `:reverse` установлен, то копируемая зона, а именно последовательность объектов данных, копируется в таблицу выхода в обратном порядке. Параметр `:reverse` действует только, когда копирование осуществляется из одномерной таблицы в одномерную или из двумерной в двумерную.

Если `:in` - двумерная таблица, то строки `:elemcount` (объекты данных) будут скопированы, начиная со строки, обозначенной `:inind`. Из каждой строки: `size(2)` будут скопированы части элементов, начиная с первого элемента строки (`elem(n,1) - :elem(n,:size(2))`).

Если `:elemcount` больше, чем `:in:header:size(1)`, то копируют только части объектов данных `:size(1)`.

Если `:inind + :elemcount` больше, чем `:in:header:size(1)`, то есть зона, которая должна быть скопирована, продолжается за пределы от используемой зоны таблицы входа, то исходящие объекты данных (`:inind + :elemcount - :in:header:size(1)`) берутся из начала входной таблицы. Это свойство может быть использовано для копирования кольцевой таблицы (например, тренда) в таблицу массива; причем первый объект данных кольцевого массива копируется в `:elem(1,*,*)` массива. Для этого указатель начальной точки входной таблицы `:in:header:base(1)` должен быть соединен с `:inind`.

Присоединение к выходной таблице

Если `:autosize = 0`, то максимум `:out:header:size(1) - :outind` частей объектов данных будет скопирован с входа `:in` на выход `:out`. Если `:autosize = 1`, то максимальное количество объектов данных `:out:header:dim(1) - :outind` будет скопировано с входа `:in` на выход `:out`.

Когда выход `:out` двумерный, строка (строки), копируемые с входа `:in`, будут присоединены к строке (строкам) выхода `:out`, начиная со строки, обозначенной с помощью `:outind`. Они будут присоединены к элементам каждой строки, начиная с первого элемента строки. Независимо от `:autosize`, максимум `:out:header:size(2)` элементов будет присоединено к каждой строке.

Если `:outind` больше, чем `:out:header:dim(1)`, то объекты данных не будут скопированы.

Объекты данных в таблице `:out`, которые не копируют с входа `:in`, остаются без изменений.

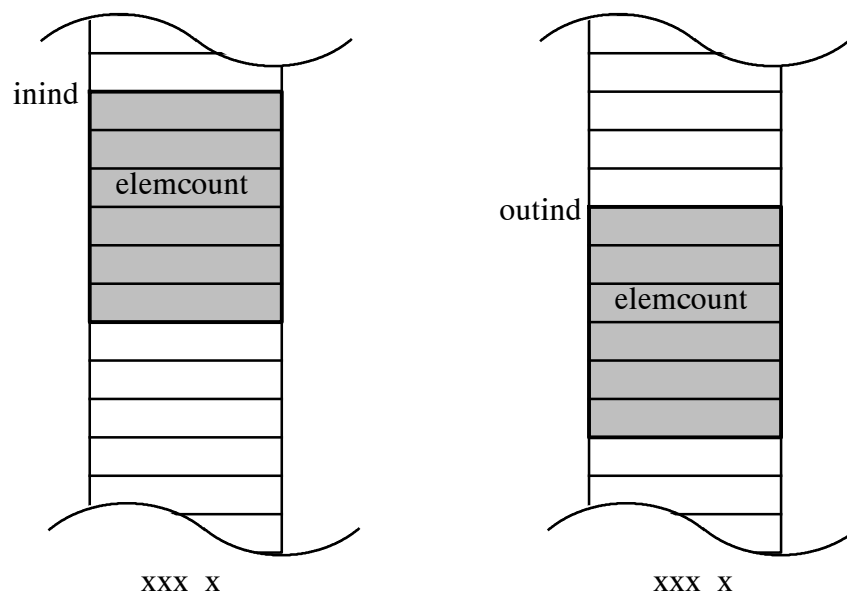


Рисунок 12 Копирование последовательности объектов данных (элементов или строк таблицы) входной таблицы в определенное место выходной таблицы

Установки после копирования

После успешного копирования поля заголовка таблицы выхода обновляют, как описано ниже, за исключением выхода таблицы `:size`. Если параметр `:autosize` есть 1 ("auto"), `:out:header:size(1)` обновляют значением `:outind + число скопированных объектов данных - 1`. В ином случае выход таблицы `:size` оставляют без изменений.

Биты ошибки `old (32)` и `inv (16)` устанавливают в неиспользованных элементах между `:size` и `:dim`. Значения элементов устанавливают на нуль или другое нейтральное значение. Выход `:ssout` также устанавливают на 1 (TRUE).

Заголовок структуры таблицы имеет следующие поля:

:fails	Указывает общие биты ошибки таблицы, которые относятся к каждому используемому элементу таблицы (части размера <code>size</code>). Кроме того, могут быть установлены другие биты ошибки в одиночных элементах таблицы.
:mode	Указывает режим таблицы ("array" или "ring").
:elemtype	Указывает имя типа рамки и элементов таблицы.
:dim	Указывает размерность рамки и максимальный размер таблицы.
:size	Указывает фактический размер таблицы.
:base	Указывает базовую точку таблицы.
:dataid	Изменение в <code>:dataid</code> указывает, что таблица была обновлена.
:time	Метка времени таблицы. Ее член <code>:bintime</code> указывает время последнего обновления таблицы.
:interval	Указывает время между последовательными элементами трендов, основанных на времени. Может также указывать промежуток между последовательными элементами (строками или столбцами) таблицы в соответствующих единицах измерения.

После успешного выполнения функциональный блок обновляет следующие поля используемой выходной таблицы:

:fails	= из входной таблицы (таблиц), маскированной <code>fmask</code>
:base	= не устанавливают
:size	= значения из входной таблицы, биты ошибки обнуляют
:dataid	= приращение на единицу
:time:binstat	= резервировано предыдущее состояние (0 или 1)
:time:bintime	= дата и время выполнения
:interval	= из входной таблицы

1.8.3 Инициализация

В цикле инициализации проверяют возможные ошибки конфигурирования (любые параметры конфигурации, выходящие из допустимого диапазона). Если ошибка конфигурирования присутствует в каком-либо параметре конфигурации, который имеет несколько допустимых значений (например, параметр `:mode`), выход `:fault` обновляют, и биты ошибки `inv` и `der` устанавливают во все элементы полей битов ошибки таблицы выхода и в поле `:header:fails`, функциональный блок не может быть выполнен прежде, чем ошибка будет исправлена. Информацию о такой ошибке хранят во внутреннем элементе функционального блока, где ее проверяют во время обычного рабочего цикла. Для параметров конфигурации, которые имеют только два допустимых значения (обычно 0 и какое-либо еще), проверку ошибок не выполняют.

Блок также проверяет, чтобы типы элементов таблицы входа и выхода и число измерений были совместимы одно с другим. Если нет, блок установит свой выход :fault и не будет продолжать выполнение.

Если нет действующих ошибок, то внутренняя функция модулей выполняется, если значение :mode равно 0 или 4, а условие выполнения Истина (true). Копирование всегда выполняется, когда значение :mode равно 6-9; значения входа :in во время цикла инициализации хранят как основные значения, с которыми сравнивают значения обычных циклов, чтобы проверить отсутствие изменений.

1.8.4 Пример конфигурации

См. пример конфигурации в главе 1.6 “Пример конфигурации 1”.

1.8.5 Структура данных

Параметры конфигурации

mode

Тип:	int16
Значение по умолчанию:	0
Описание:	Режим копирования
0 =	копировать, когда :cnd = 1
1 =	копировать по переднему фронту входа :cnd
2 =	копировать по заднему фронту входа :cnd
3 =	копировать, когда :cnd изменяется
4 =	копировать, когда :cnd = 0
5 =	недопустимо в этом функциональном блоке; если используют, интерпретируется как состояние ошибки
6 =	копировать, если :in:header:size(1) или :in:header:dataid изменяется; биты ошибки игнорируют
7 =	режим 1 или режим 6
8 =	режим 2 или режим 6
9 =	режим 3 или режим 6

Если значение отличается от 0-4, параметры 6-9 конфигурируют в режиме параметра, затем функциональный блок установит биты ошибки inv и deg во все поля битов ошибки элементов таблицы выхода и в поле бита ошибки заголовка. Выход :fault обновляют.

fmask

Тип: uns16
Значение по умолчанию: 0
Описание: Биты ошибки элемента данных каждой таблицы и заголовка могут быть маскированы во время копирования

Биты ошибки могут быть маскированы следующим значениями :fmask:

0 = маскирования нет
 2 = бит маски ext
 4 = бит маски ovf
 8 = бит маски sim
 16 = бит маски inv
 32 = бит маски old
 64 = бит маски der
 128 = бит маски sex

Несколько битов ошибки могут быть маскированы суммированием соответствующих значений.

reverse

Тип: uns16
Значение по умолчанию: 0
Описание: Реверсирование элементов в одномерной таблице и строк в двумерной таблице

0 = копировать в обычном порядке
 1 = копировать в обратном порядке

Любые другие значения приравнивают к единице. Параметр :reverse действует, только когда копируют из одномерной таблицы в одномерную или из двумерной таблицы в двумерную.

autosize

Тип: uns16
Значение по умолчанию: 0
Описание: Метод расчета размера таблицы выхода

0 = размер таблицы выхода оставляют таким, как он есть после выполнения блока
 1 = в одномерной таблице :size(1) есть :outind + число копируемых элементов - 1;
 в двумерной таблице :size(1) есть :outind + число копируемых строк - 1

Любое другое значение приравнивают к единице.

Параметры соединения**Входы****in**

Тип: table
Значение по умолчанию:
Описание: Таблица входа, тип определяют во время конфигурирования

Таблица может быть любой одномерной или двумерной. Допустимые режимы (:mode) - array и ring. Все режимы (:mode) таблицы входа обрабатывают как :mode ring.

Вход `:in` связан с элементом `:header` таблицы входа, которая должна быть сконфигурирована как интерфейсный порт или локальная точка данных.

ВНИМАНИЕ!

Внешние устройства не могут быть соединены непосредственно с входом блока. Они должны быть соединены также с интерфейсом или с локальной точкой данных того же типа.

cnd

Тип:	bin
Значение по умолчанию:	0
Описание:	Условие копирования, см. <code>:mode</code>

inind

Тип:	intl
Значение по умолчанию:	0 1
Описание:	Индекс таблицы входа для указания начала копирования

Это индекс элемента в одномерной таблице и индекс строки в двумерной таблице. Часть бита ошибки (`:inind:f`) опускают и используют абсолютные индексы.

Если `:inind` меньше или равен нулю, копирование начинается с первого элемента (для одномерной) или строки (для двумерной) таблицы входа.

outind

Тип:	intl
Значение по умолчанию:	0 1
Описание:	Индекс таблицы выхода для указания, куда поместить копию

Это индекс элемента в одномерной таблице и индекс строки в двумерной таблице. Часть бита ошибки (`:outind`) пропускают и используют абсолютные индексы.

Если `:outind` меньше или равен нулю, первым целевым элементом копирования является первый элемент (для одномерной) или строка (для двумерной) таблицы выхода.

elemcount

Тип:	intl
Значение по умолчанию:	0 0
Описание:	Число элементов, которые должны быть скопированы в одномерной таблице, и число строк в двумерной таблице

Если `elemcount` меньше или равно нулю, копируют все элементы таблицы или строки, начинающиеся от `:inind` до `:in:header:size`. Часть бита ошибки (`:elemcount:f`) пропускают.

Выходы**out**

Тип: table
Значение по умолчанию:
Описание: Таблица выхода, тип элемента такой же как таблицы входа

Таблицы array и ring будут обработаны как array. Другие спецификации см. :in.

ccout

Тип: bin
Значение по умолчанию: 48
Описание: Условие копирования выполнено

Значение 1 (TRUE) указывает, что условие копирования выполнено.

fault

Тип: bin
Значение по умолчанию: 48
Описание: Обнаружена ошибка во время выполнения

Значение 1 (TRUE) указывает, что действует, по меньшей мере, одна такая ошибка, которая мешает нормальному выполнению функционального блока.